CentraleSupélec

Developing simulation and statistical evaluation tools for a non-linear mixed-effect model

Jean-Vincent Martini

CentraleSupélec - Mention Sciences des données et de l'information &
Filière Métiers de la Recherche
Master MVA

April – September 2026

Academic Supervisor:
SDI: Arthur Tenenhaus
arthur.tenenhaus@centralesupelec.fr
FMR: Vincent Mousseau
vincent.mousseau@centralesupelec.fr
MVA: Raphaël Porcher
raphael.porcher@aphp.fr

Company Supervisor:
Sophie Tezenas du Montcel
sophie.tezenas@aphp.fr

Abstract

This internship was carried out within the ARAMIS team at the Paris Brain Institute (ICM) and focused on the development of simulation and statistical evaluation tools for Leaspy, an open-source Python library dedicated to the analysis of multivariate longitudinal data in neurodegenerative diseases. Leaspy relies on non-linear mixed-effects models to reconstruct disease progression trajectories, align patients on a common latent disease timeline, and estimate individual progression parameters from repeated clinical observations.

The main objective of this work was to extend the simulation capabilities of Leaspy to its joint model, which simultaneously models longitudinal outcomes and time-to-event data through a shared latent disease age. While simulation tools already existed for the standard logistic model, no integrated implementation was available for the joint model. The internship therefore involved designing and implementing a simulation pipeline compatible with this framework, including event-time generation, management of censoring, support for competing events, and integration with the library's existing `estimate` and `simulate` interfaces.

A second major contribution consisted in developing a validation framework based on repeated simulation and re-estimation, in order to assess the self-consistency of the implementation and the statistical recovery of both population and individual parameters. This evaluation made it possible to identify which parameters are robustly recovered and which remain difficult to estimate in moderate-sample regimes, especially in the survival and latent-source components of the model.

ATTENTION AUX PARTIES CS ONLY: RECUL ETHIQUE, GRILLE DE COMPETENCES, etc. POSSIBLEMENT A RAJOUTER ENSUITE.

CS ONLY: Résumé

Ce stage a été réalisé au sein de l'équipe ARAMIS de l'Institut du Cerveau (ICM) et a porté sur le développement d'outils de simulation et d'évaluation statistique pour Leaspy, une bibliothèque Python open source dédiée à l'analyse de données longitudinales multivariées dans le contexte des maladies neurodégénératives. Leaspy s'appuie sur des modèles mixtes non linéaires afin de reconstruire des trajectoires de progression, d'aligner les patients sur une échelle temporelle latente commune (âge de maladie) et d'estimer des paramètres individuels de progression à partir d'observations cliniques répétées.

L'objectif principal du travail a été d'étendre les capacités de simulation de Leaspy à son modèle conjoint, qui décrit simultanément l'évolution longitudinale des variables cliniques et des données de temps avant événement via une variable latente partagée. Bien que des outils de simulation existaient déjà pour le modèle logistique standard, aucune implémentation intégrée n'était disponible pour le modèle conjoint. Le stage a ainsi consisté à concevoir et implémenter une chaîne de simulation compatible avec ce cadre, incluant la génération des temps d'événement, la prise en compte de la censure, le support d'événements concurrents, ainsi que l'intégration avec les méthodes `estimate` et `simulate` de la librairie.

Une seconde contribution majeure a été le développement d'un cadre de validation basé sur des cycles répétés de simulation puis de ré-estimation, afin de vérifier l'autocohérence de l'implémentation et la capacité du modèle à retrouver les paramètres de population et individuels. Cette évaluation a permis d'identifier les paramètres estimés de manière robuste et ceux plus difficiles à récupérer en régime d'effectifs modérés, en particulier dans les composantes de survie et de sources latentes du modèle.

Acknowledgements

I would like to thank Sophie Tezenas du Montcel, team leader and scientific director of this internship, for welcoming me into her research group and for her guidance throughout the project.

I also thank my supervisors Maylis Tran and Sofia Kaisaridi for their availability, their constructive feedback during our weekly meetings, and their support on both the scientific and software development aspects of the work.

I also wish to thank the members of the ARAMIS team for the stimulating and interdisciplinary working environment they foster. The exchanges I had with researchers, engineers, and fellow interns enriched both my understanding of computational neuroscience and my daily experience at the institute.

On the academic side, I would like to thank my school tutors, Arthur Tenenhaus (SDI) and Vincent Mousseau (FMR), for their follow-up and advice throughout the internship.

Finally, I extend my thanks to the Paris Brain Institute (ICM) and INRIA for providing the institutional and computational resources that made this work possible, as well as to the open-source contributors of the Leaspy library, whose prior work laid the foundations for my own contributions.

Contents

1 Introduction

This end-of-studies internship was carried out from April to September 2026 at the Paris Brain Institute (ICM), located at the Pitié-Salpêtrière Hospital in Paris. The ICM is an internationally recognized research center dedicated to the understanding and treatment of neurological and psychiatric diseases. Its structure brings together clinicians, biologists, statisticians, computer scientists and engineers within a highly interdisciplinary environment, with the objective of fostering close interactions between methodological innovation and clinical applications.

The internship took place within the ARAMIS team (Algorithms, Models and Methods for Images and Signals), a joint team between the ICM and INRIA. ARAMIS develops computational and statistical tools for the analysis of brain data, with a particular focus on modeling disease progression in neurological disorders such as Alzheimer's disease, Parkinson's disease, Huntington's disease or amyotrophic lateral sclerosis. More specifically, I joined the longitudinal modeling group, whose research focuses on non-linear mixed-effects models for multivariate longitudinal data. This group combines expertise in statistics, machine learning and neuroscience to design models capable of capturing the temporal dynamics and inter-individual heterogeneity that characterize disease progression.

In this context, the internship was centered on Leaspy (LEArning Spatiotemporal Patterns in Python), an open-source Python library developed by ARAMIS and its collaborators for the statistical analysis of longitudinal clinical data. Leaspy aims at reconstructing disease trajectories from repeated patient measurements, aligning individuals onto a common disease timeline, and producing clinically interpretable quantities such as subject-specific disease stage or progression speed. The library already provides a mature implementation of several models and algorithms for fitting, personalization, trajectory estimation and, in some cases, simulation. However, as the methodological scope of Leaspy has expanded, especially with the recent development of a joint model linking longitudinal outcomes and survival data, the need for robust software tools for simulation and statistical evaluation has become increasingly important.

Indeed, simulation plays a central role in the development of complex statistical models. Unlike real clinical datasets, simulated datasets provide access to the ground truth: the true parameter values and latent variables are known by construction. They therefore make it possible to assess whether an estimation procedure is unbiased, whether individual parameters are accurately recovered, and whether an implementation behaves as expected under controlled conditions. In the specific case of Leaspy, this need is even stronger because the models are non-linear, involve latent random effects, and rely on computationally intensive estimation procedures such as MCMC-SAEM. Extending simulation capabilities to the joint model and designing tools to evaluate its statistical behaviour are thus both methodological and practical priorities.

The main objective of this internship was therefore to contribute to the development of simulation and statistical evaluation tools for Leaspy, with a particular focus on the joint model. Concretely, the work consisted in extending the simulation pipeline to settings involving longitudinal data jointly modeled with time-to-event outcomes, integrating these developments into the existing software architecture, and validating them through dedicated numerical experiments. Beyond the implementation effort itself, the internship also

aimed at assessing the self-consistency of the model and estimation procedures through simulation-based validation studies.

More broadly, this work lies at the intersection of statistical modeling, scientific computing and software engineering. Because Leaspy is an open-source research library intended to be used both by methodologists and by applied researchers, contributing to it requires not only implementing new methodological features, but also ensuring code reliability, reproducibility, usability and documentation quality. In that respect, the internship included several additional contributions that are not detailed in the core chapters of this report: reviewing pull requests submitted to the library, writing and improving unit and functional tests, improving parts of the documentation, and preparing and participating at a workshop on longitudinal data with a particular emphasis on simulation within Leaspy. These activities were an important part of the internship, as they contributed to the overall robustness, maintainability and diffusion of the software.

This report focuses primarily on the methodological and technical contributions related to simulation and evaluation for the joint model. It first presents the scientific and software context of the Leaspy library, as well as the motivations for simulation studies in this framework. It then introduces the joint model in more detail, describes the simulation algorithm implemented during the internship and the adaptations made within Leaspy, and finally reports a validation study designed to assess parameter recovery and the consistency of the overall pipeline.

2 Context on the Leaspy library

Neurodegenerative diseases are characterized by a slow and complex evolution, involving dozens to hundreds of heterogeneous clinical and biological markers such as continuous scores, discrete or ordinal assessments, censored survival outcomes, and high-dimensional modalities such as structural MRI or PET imaging, that evolve jointly over time. This complexity is further compounded by substantial inter-individual heterogeneity: patients sharing the same diagnosis can exhibit markedly different symptom profiles, progression rates, and responses to treatment. Modeling and predicting this progression from multivariate longitudinal data is therefore a central challenge for clinical research and precision medicine.

In this context, Leaspy (LEArning Spatiotemporal Patterns in Python), an open-source Python library, was developed by the ARAMIS team at the Paris Brain Institute (ICM) in collaboration with INRIA, INSERM, CNRS and Sorbonne University to analyze the joint evolution of numerous markers simultaneously. The repository is available at `https://github.com/aramis-lab/leaspy` and the documentation at `https://leaspy.readthedocs.io/en/stable/index.html`.

The package is designed for the statistical analysis of longitudinal medical data consisting of repeated observations of patients at different time points. Its principal use cases include: reconstructing the long-term spatiotemporal trajectory of disease evolution from short-term cohort data; positioning individual patients relative to a group-average disease timeline; quantifying the impact of cofactors (sex, genetic mutations, environmental factors) on progression; imputing missing values; predicting future observations; and simulating virtual patient cohorts.

2.1 Mathematical Background

2.1.1 Nonlinear Mixed-Effects Models

Leaspy is based on the non-linear mixed-effects models framework, which decomposes variability in longitudinal observations into two complementary components [17]:

- **Fixed effects** (population-level): systematic trends shared across all individuals, such as the average disease trajectory, typical progression speed, and population reference time.
- **Random effects** (individual-level): subject-specific deviations capturing heterogeneity in onset time, progression speed, and the relative ordering of biomarker changes.

Formally, let y denote the observed data, $z = (z_{\text{fe}}, z_{\text{re}})$ the latent fixed and random effects, θ the model parameters, and Π the known hyperparameters. Parameter estimation is performed by maximizing the marginal likelihood

$$p(y \mid \theta, \Pi) = \int_z p(y, z \mid \theta, \Pi)\, dz, \tag{1}$$

which decomposes in log-space as

$$\log p(y, z \mid \theta, \Pi) = \underbrace{\log p(y \mid z, \theta, \Pi)}_{\text{data attachment}} + \underbrace{\log p(z_{\text{re}} \mid z_{\text{fe}}, \theta, \Pi) + \log p(z_{\text{fe}} \mid \theta, \Pi)}_{\text{prior attachment}}. \tag{2}$$

Because this integral is intractable in closed form, Leaspy employs the MCMC-SAEM algorithm (Markov Chain Monte Carlo Stochastic Approximation Expectation-Maximization) [8]. This algorithm alternates between (i) a stochastic approximation step that uses Gibbs sampling to draw latent variables conditional on current parameter estimates, and (ii) a maximization step that re-estimates fixed effects and variance components from the updated sufficient statistics.

Observed scores are further modelled with additive noise of standard deviation σ, a population parameter estimated jointly with the trajectory parameters.

2.1.2 Riemannian Geometry of Disease Progression

A key innovation of Leaspy is the interpretation of disease progression within a Riemannian geometric framework [17]. The core idea is to represent the multivariate progression of clinical markers as a geodesic on a product Riemannian manifold, where the shape of the curve is entirely determined by the choice of the metric $G(p)$.

For clinical scores exhibiting floor or ceiling effects, common in neurodegenerative disease assessment, the manifold $(0, 1)$ equipped with the metric $G(p) = 1/[p^2(1-p)^2]$ yields logistic (sigmoidal) trajectories. This metric encapsulates the bounded nature of the scores in a geometrically principled way. More generally, the Euclidean metric on $\mathbb{R}^n$ yields linear trajectories, allowing the framework to accommodate diverse data types.

The average population trajectory γ_0 is a geodesic parameterized by its initial conditions at reference time t_0: initial position $p_0 = \gamma_0(t_0)$ and initial speed $v_0 = \dot{\gamma}_0(t_0)$. For numerical convenience, the model internally works with the unconstrained reparametrisations $\tilde{g} = \text{logit}(p_0)$ and $\tilde{v}_0 = \log v_0$, which lift the constraints $p_0 \in (0, 1)$ and $v_0 > 0$ and constitute the quantities directly estimated by MCMC-SAEM. Individual trajectories $\gamma_i(t)$ are derived from this average through three types of random effects:

- **Time shift** $\tau_i \sim \mathcal{N}(t_0, \sigma_\tau^2)$: earlier or later disease onset relative to the population average, where σ_τ is the inter-individual standard deviation of onset times.
- **Log-acceleration factor** $\xi_i \sim \mathcal{N}(0, \sigma_\xi^2)$: faster or slower progression speed, where σ_ξ is the inter-individual standard deviation of log-acceleration factors.
- **Space shifts** $w_{i,k}$: modifications to the relative ordering and timing of individual biomarker progressions, encoded as tangent-space perturbations.

These three effects are combined through a latent disease age reparametrization:

$$\psi_i(t) = v_0 \, e^{\xi_i}(t - \tau_i) + t_0, \tag{3}$$

which transforms the calendar age t of patient i into a disease-stage age. This reparametrization provides a biologically interpretable decomposition: τ_i captures onset heterogeneity, e^{ξ_i} captures progression rate heterogeneity, and the space shifts $w_{i,k} = (\mathbf{A}\mathbf{s}_i)_k$, obtained via the mixing matrix $\mathbf{A}$ acting on independent sources $\mathbf{s}_i$, capture differences in the sequential ordering of biomarker changes across patients. The columns of $\mathbf{A}$ are constructed from a set of free coefficients β, jointly estimated with the other population parameters during model fitting.

2.2 Main Models

Logistic Model. The logistic model is the core model of Leaspy, designed for multivariate longitudinal data whose components exhibit sigmoidal (bounded, monotone) progressions, typical of cognitive scores or normalized biomarkers. Such S-shaped trajectories naturally capture the characteristic three-phase pattern observed in many neurodegenerative diseases: an initial slow deterioration when the disease is subclinical, a rapid inflection phase corresponding to the onset of clinically detectable decline, and a final plateau as scores approach their floor or ceiling values.

Joint Model. The joint model [14, 15] extends the logistic model to simultaneously analyze longitudinal measurements and time-to-event data (survival, clinical events, competing risks). Rather than treating these processes separately, which can lead to biases from informative dropout, the joint model links them through the shared latent disease age $\psi_i(t)$, avoiding the need to define an explicit disease reference time. It is on this model that the simulation and evaluation tools developed in this work are focused.

Mixture Model. Standard mixed-effects models assume a single population trajectory with random individual deviations. The mixture model [5] relaxes this assumption by modeling the population as a combination of n_c latent subgroups, each with its own distribution of individual parameters. This formulation enables simultaneous disease progression modeling and probabilistic patient subtyping.

2.3 Main Algorithms

Leaspy provides four main algorithmic operations:

`fit`: Estimates population parameters (fixed effects and variance components) from a cohort of longitudinal observations using MCMC-SAEM. The user specifies the model type, number of outcomes, number of latent sources, and observational noise model. The result is a fitted model characterizing the average disease trajectory and its population-level variability.

`personalize`: Given a fitted model, estimates individual random effects $(\tau_i, \xi_i, \mathbf{s}_i)$ for each subject from their available observations. Two approaches are available: a frequentist mode using `scipy.optimize.minimize` to maximize the individual likelihood, and a Bayesian mode using a Gibbs sampler to extract the mean or mode of the posterior distribution. The output is a set of individual parameters that characterize each patient's position on the disease timeline.

`estimate`: Evaluates the modeled trajectory $\gamma_{i,k}(t)$ for a specific patient at arbitrary time points, using the previously estimated individual parameters. This enables reconstruction of the full disease trajectory, imputation of missing values, and forecasting of future clinical scores.

`simulate`: Generates synthetic patient cohorts by (i) sampling individual parameters from the learned population distributions, (ii) generating visit times according to user-specified schedules (random, regular, or custom), and (iii) evaluating model trajectories with added observational noise. This functionality is only available for the logistic model for now, but a part of this work consists in extending it to the joint model.

2.4 Applications and Related Work

2.4.1 Applications to Neurodegenerative Diseases

Leaspy has been applied to neurodegenerative diseases, demonstrating its generalizability and clinical relevance.

Parkinson's disease. Leaspy comes with a built-in Parkinson's disease dataset, enabling out-of-the-box demonstration and benchmarking. Figure 1 illustrates the typical results obtained with the software.

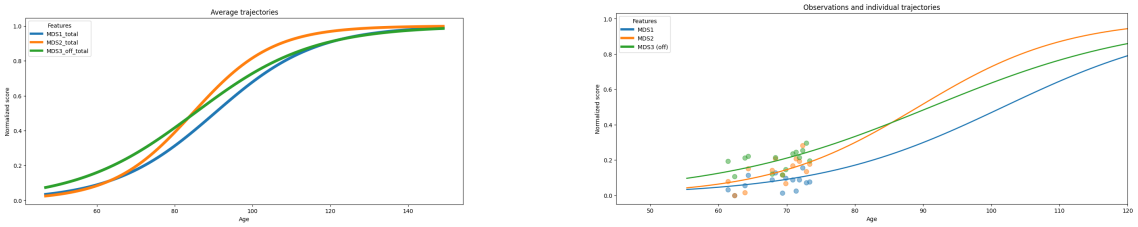

Figure 1: Disease progression modeling with Leaspy on the built-in Parkinson's dataset. **Left:** Average population trajectory of clinical scores over the estimated disease timeline. **Right:** Personalized trajectory (solid line) and observed visits (dots) for one patient.

Alzheimer's disease (AD). The AD Course Map [6] uses Leaspy to create a spatiotemporal atlas of AD progression by jointly modeling neuropsychological assessments, the propagation of hypometabolism and cortical thinning across brain regions, and the deformation of the hippocampal shape. It outperformed 56 competing methods in the TADPOLE open challenge [12] and, when tested globally on over 4,600 patients from ADNI, PharmaCog, and INSIGHT-preAD, demonstrated the ability to enrich clinical trials with predicted progressors, reducing required sample sizes by 38% to 50% depending on trial duration, outcome measure, and targeted disease stage [11].

Huntington's disease. Koval et al. applied the disease course mapping framework to identify the optimal intervention window for Huntington's disease clinical trials, demonstrating that the model could "spot the time to treat" by positioning patients on the disease timeline prior to symptom onset [7].

CADASIL. Kaisaridi et al. applied Leaspy to 395 patients from the French CADASIL referral center, modeling the joint progression of 14 clinical scores. Using a probabilistic

mixture framework integrated into the MCMC-SAEM estimation procedure, they identified two clinically meaningful subtypes of disease progression: an early-onset, rapidly progressing group and a late-onset, slowly progressing group. This was achievedwithout resorting to post-hoc clustering, providing new insights into the heterogeneity of this rare cerebral small-vessel disease [5].

Amyotrophic lateral sclerosis (ALS). The joint cause-specific spatiotemporal model was validated on data from the PRO-ACT dataset (over 2,100 ALS patients). The model jointly analyzes three ALSFRS-R subscales (bulbar, fine motor, gross motor) with competing events (non-invasive ventilation initiation and death) through a shared latent disease age. On a 10-fold cross-validation benchmark, the model achieved systematically better event discrimination (AUC, C-index) than the standard shared-random-effect joint model (JMbayes2), while using fewer parameters and avoiding the need for a precise reference onset time [15].

2.4.2 Related approaches.

Leaspy belongs to the broader family of data-driven disease progression models, a field that has developed rapidly since the early 2010s [16, 21]. These methods can be organized along two axes: the type of data they require (cross-sectional vs. longitudinal) and whether they estimate a single disease timeline or multiple subtypes.

Cross-sectional models. The Event-Based Model (EBM) [3] and its discriminative variant (DEBM) [18] infer a cumulative sequence of biomarker abnormality events from a single visit per individual. While data-economical, they produce only discrete severity orderings and have limited forecasting utility without additional longitudinal information. The Subtype and Stage Inference algorithm (SuStaIn) [20] augments the EBM concept with unsupervised clustering, enabling joint subtyping and staging from cross-sectional data alone. SuStaIn has yielded high-impact results across Alzheimer's disease [19], multiple sclerosis [2], and other conditions, but does not model continuous biomarker trajectories.

Longitudinal models. Several latent-time mixed-effects models have been proposed for continuous trajectory estimation. The Disease Progression Score (DPS) [4] aligns biomarker data via a linear transformation of age into a disease score with sigmoid group-level dynamics; the Latent Time Joint Mixed Model (LTJMM) [9] extends this with a flexible Bayesian framework and covariate support. The Gaussian Process Progression Model (GPPM) [10] offers a nonparametric alternative that avoids imposing parametric trajectory shapes but can be computationally expensive on large cohorts. SubLign [1] combines deep generative models with latent time-shifts for simultaneous subtyping and trajectory alignment using longitudinal data.

Compared to these approaches, Leaspy's Disease Course Mapping framework, introduced by Schiratti et al. [17] and extended by Koval et al. [6], offers a distinctive combination of features: (i) a Riemannian geometric formulation that naturally handles bounded scores via logistic geodesics, (ii) interpretable individual-level random effects decomposed

into time shift, acceleration, and inter-biomarker space shifts, (iii) a joint model linking longitudinal outcomes and competing survival events through a shared latent disease age [15], and (iv) a mixture extension for unsupervised subtyping [5].

3 Motivation for Simulation Studies and Tools

3.1 The Role of Simulation in Statistical Methodology

Simulation studies occupy a central position in the development and validation of statistical methods. Their fundamental advantage is epistemological: because the data-generating process is entirely under the experimenter's control, the true values of all parameters and latent quantities are known by construction. This makes it possible to evaluate estimator properties such as bias, variance, coverage of confidence intervals, sensitivity to model misspecification, that are inaccessible on real data, where a ground truth is never observed. As Morris, White and Crowther emphasize in their reference tutorial on the design, conduct and reporting of simulation studies [13], this ability to compare estimates against a known truth is the defining strength of the simulation approach, and what distinguishes it from empirical benchmarks on real-world datasets.

Despite their power, simulation studies are often poorly designed, inadequately analyzed and incompletely reported, which can severely limit their scientific value and reproducibility. The ADEMP (Aims, Data-generating mechanisms, Estimands, Methods, and Performance measures) framework proposed by Morris et al. [13] provides a principled template for conducting and communicating such studies. This framework guided the simulation study reported in the original joint model paper [15], and it underpins the design choices made in the simulation tools developed in the present work.

Beyond methodological validation, simulated datasets present a practical advantage that is increasingly relevant in clinical research: they can be freely shared and distributed without the ethical, legal or privacy constraints that govern real patient data. A synthetic cohort generated under a calibrated model preserves the statistical properties of the original population (trajectory shapes, individual variability, censoring patterns) while containing no identifiable patient information. This makes simulated data a natural vehicle for reproducible research, open benchmarking, and collaborative method development across institutions.

3.2 Importance in the Leaspy Context

The combination of model complexity, data structure, and estimation challenges that characterizes the Leaspy framework makes simulation studies methodologically indispensable. Several distinct considerations motivate this claim.

Absence of ground truth on real data. On real longitudinal cohorts such as PROACT (ALS) or ADNI (Alzheimer's disease), the true individual random effects $(\tau_i, \xi_i, \mathbf{s}_i)$ are never observed. One can fit a model and obtain estimated parameters, but there is no external criterion against which to verify whether the estimates are biased, whether the posterior is well calibrated, or whether the algorithm has converged to the correct solution. Simulation is the only means to construct a controlled setting in which the estimator's output can be directly compared to a known truth, and thus the only means to quantify bias, mean squared error, and interval coverage.

Comparing personalization strategies without an oracle. Once the population model has been fitted, individual random effects are recovered via a `personalize` step that offers two algorithmic modes: a frequentist MAP estimate obtained by numerical optimization, and a Bayesian posterior estimate obtained by Gibbs sampling. These two approaches differ in computational cost, in the uncertainty information they return, and potentially in the accuracy of their point estimates. In clinical applications, biases in the estimated time shift $\hat{\tau}_i$ directly affect the positioning of each patient on the disease timeline, with consequences for trial enrichment, progression forecasting, and biomarker ordering analyses. Determining in which regimes each approach is preferable requires a ground truth that only simulation can supply.

Cases with sparse and heterogeneous data. Patients with neurodegenerative diseases often contribute a highly variable number of visits: some are followed for only two or three appointments, others for a decade or more, with irregular and patient-specific inter-visit intervals. This can lead to trajectories that are poorly identified from short follow-ups. In this regime of sparse observations, classical asymptotic guarantees such as consistency or normality of maximum likelihood estimators are not reliable guides to finite-sample behavior. Simulation provides the only principled way to explore how estimation quality degrades as the number of visits decreases, as visit spacing increases, or as individual heterogeneity grows.

Informative dropout and its interaction with the model. A pervasive challenge in neurodegenerative disease cohorts is informative dropout: patients who progress most rapidly are precisely those most likely to leave the study early, either because of disease severity, hospitalization, or death. This means that the absence of data is not random but is correlated with the unobserved latent state of the patient. Ignoring this mechanism leads to biased estimates of population parameters and individual trajectories. The joint model [14, 15] is specifically designed to address this problem by linking the longitudinal trajectory and the time-to-event process through a shared latent disease age, so that informative censoring is modeled rather than ignored. Simulation is the primary tool for verifying that the implementation of the joint simulation algorithm correctly reproduces this mechanism and for confirming that the estimation procedure remains approximately unbiased under this form of dependent censoring.

Non-linearity and Riemannian geometry. For standard linear models or generalized linear models, theoretical results guarantee consistency and asymptotic normality of maximum likelihood estimators under regularity conditions. In the spatiotemporal NLMEM framework of Leaspy, which operates on a product Riemannian manifold and involves intractable marginal likelihoods approximated by MCMC-SAEM, these guarantees are far harder to establish formally. Whether the algorithm converges to the correct fixed point, how many SAEM iterations are needed, and how sensitive the estimates are to initialization are all empirical questions that cannot be resolved analytically in the regime of real cohort sizes. Simulation studies allow these questions to be examined directly, under conditions that mirror the application setting.

Prerequisite for downstream statistical analyses. The individual parameters estimated by Leaspy will also be used as inputs to secondary analyses: variable selection, explainability methods, or predictive modeling applied to the personalized parameters $\hat{\tau}_i$, $\hat{\xi}_i$, and $\hat{\mathbf{s}}_i$. If these estimates are systematically biased or poorly calibrated, any downstream method, regardless of its own statistical validity, will inherit and propagate these errors, producing findings that cannot be reliably interpreted. Validating the estimation procedure through simulation is therefore a methodological prerequisite for any conclusion drawn from the individual parameters.

3.3 Objectives of the Simulation Tools Developed in This Work

The present work develops and integrates into the Leaspy codebase a set of simulation tools specifically designed for the joint model. These tools serve two complementary purposes.

The first is algorithmic: to provide a self-consistent and numerically robust implementation of the joint simulation procedure, extending the existing `simulate` method, currently restricted to the logistic model, to the joint model with its competing-events variant. This required extending the `estimate` method of the joint model to return event predictions compatible with the simulation pipeline, implementing inverse-transform sampling on the discrete visit grid, and handling the generalization to competing events through cause-specific conditional cumulative incidence functions.

The second is methodological: to enable rigorous simulation studies of the joint model, following the ADEMP framework [13]. By generating synthetic cohorts under known model parameters, calibrated to real-world datasets such as PRO-ACT, these tools make it possible to evaluate the bias and coverage of MCMC-SAEM parameter estimates, to compare MAP and Gibbs personalization under varying data densities, and to quantify the effect of informative dropout on estimation. They also produce shareable, privacy-preserving synthetic datasets that can be distributed alongside the model code for reproducibility and benchmarking purposes.

4 Details on the Joint Model

The joint model [15] extends the Leaspy longitudinal framework to simultaneously model repeated clinical measurements and survival outcomes, without requiring a reliable reference time point. This is a key advantage in neurodegenerative disease contexts where disease onset is not directly observable.

The model is composed of three interconnected components. The longitudinal sub-model describes the evolution of a clinical score via a logistic trajectory parameterized by the latent disease age $\psi_i(t)$ (Equation (3)), as already introduced in the main Leaspy framework. The survival sub-model describes the probability that patient i survives beyond time t using a Weibull distribution applied to the latent disease age:

$$S_i(t) = S_0(\psi_i(t)) = \mathbf{1}_{\psi_i(t)>t_0} \exp\left(-\left(\frac{\psi_i(t) - t_0}{\nu}\right)^{\rho}\right) + \mathbf{1}_{\psi_i(t)\leq t_0}, \tag{4}$$

where $\nu > 0$ is a scale parameter and $\rho > 0$ is the shape parameter of the Weibull distribution. The model estimates the reparametrisations $\tilde{\nu} = -\log \nu$ and $\tilde{\rho} = \log \rho$, which enforce positivity while enabling unconstrained optimisation. The linkage structure connecting the two sub-models is the shared latent disease age $\psi_i(t) = e^{\xi_i}(t - \tau_i) + t_0$, which encodes individual variability in disease onset (τ_i) and progression rate (e^{ξ_i}). This differs from classical shared-random-effects joint models, which link the survival hazard to the value of the longitudinal outcome. Here the link is structural, operating directly at the level of the disease timeline.

The formulation extends naturally to $L \geq 1$ competing events by assigning an independent pair of Weibull parameters (ν_l, ρ_l) to each event type $l \in \{1, \ldots, L\}$. In the single-event case the quantity of interest is the overall survival probability $S_i(t)$ and in the competing-events case, it becomes the set of cause-specific cumulative incidence functions $\mathrm{CIF}_{i,l}(t)$, each measuring the probability that event l occurs by time t in the presence of all other competing risks. When the longitudinal sub-model includes $K \geq 1$ latent sources $\mathbf{s}_i \in \mathbb{R}^K$, the survival sub-model is further coupled to the individual biomarker ordering pattern through a matrix of mixing weights $\boldsymbol{\zeta} \in \mathbb{R}^{K \times L}$. The survival shift $(\boldsymbol{\zeta}^\top \mathbf{s}_i)_l$ then enters an accelerated failure time reparametrization of the Weibull scale parameter for patient i and event l:

$$\nu_{i,l} = \nu_l \exp\left(-\xi_i - \frac{1}{\rho_l}(\boldsymbol{\zeta}^\top \mathbf{s}_i)_l\right), \tag{5}$$

so that the individual biomarker ordering pattern encoded in $\mathbf{s}_i$ accelerates or decelerates the event-specific hazard beyond the shared disease pace e^{ξ_i}. The weight matrix $\boldsymbol{\zeta}$ is estimated jointly with all other fixed effects during the MCMC-SAEM fitting procedure.

Estimation proceeds via the MCMC-SAEM algorithm, initialized from a pre-trained longitudinal model and a Weibull Accelerated Failure Time (AFT) survival model. The algorithm runs for a burn-in phase followed by a Robbins-Monro convergence phase to extract posterior means of the fixed effects. Individual random effects (τ_i, ξ_i) are personalized in a second step by maximizing the posterior likelihood given each patient's observations and censored event time, using `scipy.optimize.minimize`.

Both univariate and multivariate versions of the joint model were evaluated on simulated data according to the ADEMP framework [13]. For the univariate Joint Temporal model, $M = 100$ datasets of $N = 200$ patients were generated under the model itself, using ALS-calibrated parameters derived from the PRO-ACT dataset. In this setting, the estimation procedure recovered longitudinal fixed effects and random-effect variances with RRMSE below 11%, whereas the survival fixed effects were estimated less precisely, with RRMSE reaching 21% for the Weibull shape parameter. Coverage rates remained above 80% on average, although for about half of the parameters the associated 95% confidence intervals did not include the nominal 95% level. At the individual level, the random effects were nonetheless well recovered, with intraclass correlations of 0.96 for the time shift (τ) and 0.92 for the log speed factor (ξ).

For the multivariate Joint cause-specific Spatiotemporal model, $M = 100$ datasets of $N = 300$ patients were simulated with four longitudinal outcomes and two competing events (NIV initiation and death). Across the 19 fixed effects, the relative bias remained below 20% in absolute value and the RRMSE below 25%. In addition, 18 out of 19 coverage-rate intervals contained the target 80% level. The individual random effects were also satisfactorily recovered, with intraclass correlations above 0.84 for all components except the survival shifts, for which estimation was more difficult because of the limited number of observed events.

Both models were then benchmarked on real ALS data from the PRO-ACT dataset. The univariate model was compared with four reference approaches: a Weibull AFT survival model, the standalone longitudinal model, a two-stage model, and a shared-random-effects joint model implemented in JMbayes2. It achieved better survival discrimination than JMbayes2, with a mean cumulative AUC of 0.67 versus 0.61 ($p = 1.7 \times 10^{-3}$), and a slightly lower longitudinal MAE (4.21 versus 4.24 ALSFRS-R points, $p = 1.4 \times 10^{-17}$). The standalone longitudinal model, however, obtained the lowest MAE at 4.18 (4.38) ($p = 2.2 \times 10^{-73}$). The multivariate model reached a level of overall performance comparable to that of the cause-specific JMbayes2 model, while providing a more parsimonious representation of latent disease age and of the effect of longitudinal-outcome ordering on event occurrence. Full details of these simulation studies and benchmark experiments are reported in [14, 15].

5 The Joint Simulation Algorithm

5.1 The Original Algorithm

In the original article describing the joint model [15], the simulation of synthetic patient cohorts under the joint model proceeds in six sequential steps, described below.

1. **Simulate individual random effects**. For each patient, draw the log-acceleration factor ξ_i and time shift τ_i independently from their population distributions: $\xi_i \sim \mathcal{N}(0,\, \sigma_\xi^2), \quad \tau_i \sim \mathcal{N}(t_0,\, \sigma_\tau^2)$.

2. **Simulate age at first visit**. The age at first visit is drawn as $t_{i,0} = \tau_i + \delta_{f_i}$, where $\delta_{f_i} \sim \mathcal{N}(\bar{\delta}_f, \sigma_{\delta_f}^2)$ is a random offset relative to the patient's estimated disease onset.

3. **Simulate visit schedule**. Inter-visit intervals $\delta_{v_{i,j}} \sim \mathcal{N}(\bar{\delta}_v, \sigma_{\delta_v}^2)$ and a total follow-up duration $T_{f_i} \sim \mathcal{N}(\overline{T_f}, \sigma_{T_f}^2)$ determine the visit times $t_{i,0} < t_{i,1} < \cdots < t_{i,n_i}$, where n_i is the number of visits satisfying $t_{i,j} \leq t_{i,0} + T_{f_i}$.

4. **Simulate longitudinal outcomes**. The latent disease age $\psi_i(t_{i,j})$ is computed for each visit. The expected outcome value is then $\gamma_0(\psi_i(t_{i,j}))$ as defined by the logistic trajectory. Observed outcomes are sampled with additive noise; in the simulation study, a Beta distribution with mode $\gamma_0(\psi_i(t_{i,j}))$ and concentration p is used to match the bounded nature of clinical scores: $y_{i,j} \sim \mathcal{B}(\gamma_0(\psi_i(t_{i,j})), p)$.

5. **Simulate event times**. The event time T_{e_i} is drawn from the Weibull survival distribution implied by the latent disease age. Using the inverse-transform method, this is equivalent to: $T_{e_i} \sim e^{-\xi_i}\, \mathcal{W}(\nu, \rho) + \tau_i$, where $\mathcal{W}(\nu, \rho)$ denotes the Weibull distribution with scale ν and shape ρ.

6. **Apply censoring**. The event time and visit schedule are reconciled by applying two censoring rules: (i) all visits occurring after the event ($t_{i,j} > T_{e_i}$) are removed from the follow-up; (ii) events occurring after the last remaining visit ($T_{e_i} > t_{i,\max(j)}$) are treated as right-censored. This produces the observed tuple $(y_{i,j}, T_{e_i}, B_i^e)$, where the censoring indicator $B_i^e = 1$ if the event was observed and $B_i^e = 0$ if it was censored.

The simulation algorithm relies on two disjoint sets of parameters. The model parameters, read directly from the fitted joint model, are: the population distributions of the individual random effects (t_0, σ_τ, σ_ξ), the Weibull survival parameters (ν, ρ), and the source-to-survival mixing weights ζ whenever the model includes latent sources. These parameters are set once and for all during model fitting and are not accessible to the user at simulation time. The study parameters, supplied by the user, govern the structure of the synthetic cohort: the number of patients N, the mean and standard deviation of the first-visit offset from disease onset ($\bar{\delta}_f$, σ_{δ_f}), the mean and standard deviation of the per-patient follow-up duration ($\overline{T_f}$, σ_{T_f}), and the mean and standard deviation of inter-visit intervals ($\bar{\delta}_v$, σ_{δ_v}). An additional technical parameter, `min_spacing_between_visits`, enforces a minimum gap between successive visits and prevents numerical artefacts from excessively close observations.

5.2 Adaptations in the Leaspy Implementation

The Leaspy implementation follows the six-step procedure above and introduces the following adaptations.

Visit schedule modes. The implementation supports two distinct visit generation modes. In the *random* mode, the visit schedule is generated stochastically following steps 2 and 3 above, driven by the study parameters supplied by the user. In the *dataframe* mode, the user instead provides an explicit DataFrame specifying each patient's identifier and visit times directly, bypassing steps 2 and 3 entirely. Individual parameters are then sampled from the fitted population distributions for each patient listed in the DataFrame, in the order they appear.

Data driven estimation of study parameters. As an extension developed in this work, when the user does not supply some or all study parameters for the *random* mode, they can instead provide a dataset, usually the same dataset used to fit the model. In that case, the missing parameters are estimated automatically from the observed visit process. Specifically, $\bar{\delta}_f$ is set to zero by construction, since the empirical mean of centered first-visit ages is zero by definition; σ_{δ_f} is taken as the empirical standard deviation of observed first-visit ages across patients, which serves as a proxy for the dispersion of the first-visit offset from disease onset, given that the individual disease onsets τ_i are not directly observable. The parameters $\overline{T_f}$ and σ_{T_f} are obtained from the empirical distribution of per-patient follow-up durations (last minus first visit), and $\bar{\delta}_v$ and σ_{δ_v} are estimated from the distribution of all observed inter-visit gaps. This feature is not present in the base simulation algorithm of Leaspy for the logistic model; it introduces a dependency on the training data and breaks the strict separation between the generative mechanism and the observed sample, which should be kept in mind when interpreting simulation results.

Standardization of latent sources. When the model includes latent sources ($K \geq 1$), the source components $s_{i,k}$ for each dimension k are first drawn independently from $\mathcal{N}(0,1)$ and then standardized across the simulated cohort: each source dimension is empirically recentered and rescaled to unit variance. This ensures that the sources are approximately orthogonal to the mean in finite samples, reproducing the centering constraint that is imposed during model fitting.

Event time sampling via the adapted `estimate` method. A key contribution of this work is the extension of the joint model's `estimate` method to return event predictions alongside longitudinal trajectories, and its integration into the simulation pipeline. The original algorithm samples event times directly from the closed-form Weibull formula $T_{e_i} \sim e^{-\xi_i}\,\mathcal{W}(\nu,\rho) + \tau_i$. The implementation instead reuses the model's own `estimate` method, which evaluates the individual trajectory at each planned visit time $t_{i,j}$ and returns a conditional quantity: for a single-event model it returns the conditional survival $S_i(t_{i,j})/S_i(t_{i,0})$, and for competing events it returns the conditional cause-specific cumulative incidence functions $\mathrm{CIF}_{i,l}(t_{i,j})/S_i(t_{i,0})$, each conditioned on the patient being alive at the first planned visit $t_{i,0}$. From these, the total conditional cumulative distribution

function is assembled as:

$$F_i^{\text{cond}}(t_{i,j}) = \begin{cases} 1 - S_i(t_{i,j}) \,/\, S_i(t_{i,0}) & \text{(single event)}, \\ \displaystyle\sum_{l=1}^{L} \text{CIF}_{i,l}(t_{i,j}) \,/\, S_i(t_{i,0}) & \text{(competing events)}. \end{cases} \tag{6}$$

Inverse-transform sampling is then applied on this discrete grid: a uniform draw $U \sim \mathcal{U}[0,1]$ is compared to F_i^{cond} at each visit, and the event time is set to the first visit time $t_{i,j}$ at which $F_i^{\text{cond}}(t_{i,j}) \geq U$. If U exceeds the total conditional CDF at the last planned visit, the patient is right censored at that visit.

Two consequences follow from this design. First, because the CDF is evaluated only at the planned visit times, event times are constrained to lie on the visit grid rather than being continuous. Second, because the sampling is conditioned on survival at $t_{i,0}$, the event can never be placed before the first planned visit.

Beta noise with variance clamping. Longitudinal outcomes are sampled from a Beta distribution with mean $\mu = \gamma_0(\psi_i(t_{i,j}))$ and variance σ^2 taken from the fitted noise model. A Beta distribution with mean μ is only well defined when $\sigma^2 < \mu(1-\mu)$; this constraint can be violated when the trajectory approaches 0 or 1, that is when the patient is near the floor or ceiling of a clinical score. In such cases the implementation clamps the variance to $0.99\,\mu(1-\mu)$ and emits a warning identifying the affected patient, visit time, and feature.

Censoring after rounding. The minimum visit spacing filter operates by rounding visit times to the decimal precision compatible with `min_spacing_between_visits` and removing duplicate time points. Because rounding can advance a visit time beyond the event time, a final censoring pass is applied after rounding: any visit whose rounded time exceeds the patient's event time is removed from the dataset. This step is not part of the original six-step procedure but is required by the discrete-grid implementation to preserve the constraint that all longitudinal observations precede the event.

Generalization to competing events. In the single-event case the event indicator is set to 1 whenever the event time falls within the follow-up window. In the competing-events case ($L > 1$), the event type l^* is drawn at the identified visit time by treating the incremental cause-specific CIF as unnormalized probabilities:

$$p(l^* = l) \;\propto\; \max\left(\text{CIF}_{i,l}(t_{i,j}) - \text{CIF}_{i,l}(t_{i,j-1}),\; 0\right), \tag{7}$$

and the censoring indicator is set to $B_i^e = l^*$, following the convention that $B_i^e = 0$ denotes censoring and $B_i^e = l$ denotes cause-l event.

Test coverage in the library. The implementation is accompanied by unit and functional tests integrated into the Leaspy test suite. On the simulation side, two unit tests cover the `JointSimulationAlgorithm`: one verifies the *random* visit mode by checking that the output dataset is non-empty, contains the expected number of patients, and that visit times are strictly increasing within each individual; the other verifies the *dataframe*

visit mode by checking that all simulated visits belong to the user-supplied input schedule (the algorithm may drop visits after a simulated event, but must not introduce new time points) and that all model features are present in the output. On the `estimate` method side, four functional tests cover the extended joint model behaviour: one checks numerical correctness of the returned values for a univariate model against hardcoded expectations; one verifies that `to_dataframe=True` produces a `DataFrame` with columns `model.features + model.event_features` whose values are consistent with the raw dictionary output; one checks that passing a `pd.MultiIndex` as timepoints automatically activates the DataFrame output mode and preserves the original index ordering; and one tests the multivariate case, verifying that the output shape is $(n_{\text{timepoints}},\ n_{\text{features}}+L)$ where L is the number of events.

6 Validation and Results

6.1 Method of Evaluation

6.1.1 Principle

The evaluation rests on a simulation-based identification scheme whose conceptual core is the following: if a model is correctly specified and its estimation procedure is consistent, then fitting the model on data generated by that same model under known parameters must recover those parameters. This self-consistency requirement is the most demanding test one can impose on a complex generative pipeline, because it simultaneously exercises the simulation algorithm, the likelihood implementation, and the numerical estimation procedure. The triangular structure of the evaluation, from reference model to synthetic cohort to re-estimated model, makes it possible to verify that the model truly encodes, in its generative mechanism, the same statistical structure it learns during fitting. The procedure unfolds in three sequential stages, summarised in Figure 2.

1. **Reference model.** A reference parameter vector θ^* is defined, either by fitting the joint model on a real dataset or by setting it to a synthetic ground truth. This reference model serves as the source of truth for all subsequent comparisons.
2. **Repeated simulation and re-estimation.** A total of M synthetic cohorts are generated by the joint simulation algorithm under the reference model parameters θ^*. Each synthetic cohort is then fitted from scratch using the same MCMC-SAEM procedure, yielding M estimated parameter vectors $\hat{\theta}^{(m)}$. Individual random effects are also recovered for each synthetic patient and compared to the true values used to generate the data.
3. **Comparison.** The M estimated parameter vectors $\hat{\theta}^{(m)}$ are compared component-wise to θ^* through the scalar performance measures described in Section 6.1.2. The $N \times M$ pairs of true and estimated individual random effects are assessed through agreement statistics that distinguish systematic bias from random variability.

The diagnostic force of this design lies precisely in its circularity. The simulated data are not drawn from an independent external process: they are direct realisations of the model being validated. If the simulation algorithm correctly implements the joint model structure, and if the estimation procedure is consistent, then the estimated parameters must converge to the true ones as N and M grow. A persistent bias in $\hat{\theta}^{(m)}$, or a low intraclass correlation between estimated and true individual parameters, therefore constitutes evidence of an inconsistency between the generative mechanism and the likelihood, an identifiability failure at the chosen sample size, or a numerical artefact in the optimisation. This makes simulation-based self-consistency the primary instrument for verifying that the joint model not only fits real data but genuinely represents the underlying generative process.

6.1.2 Metrics

Two families of performance measures are used, corresponding to the two levels of inference in the joint model: population fixed effects and individual random effects.

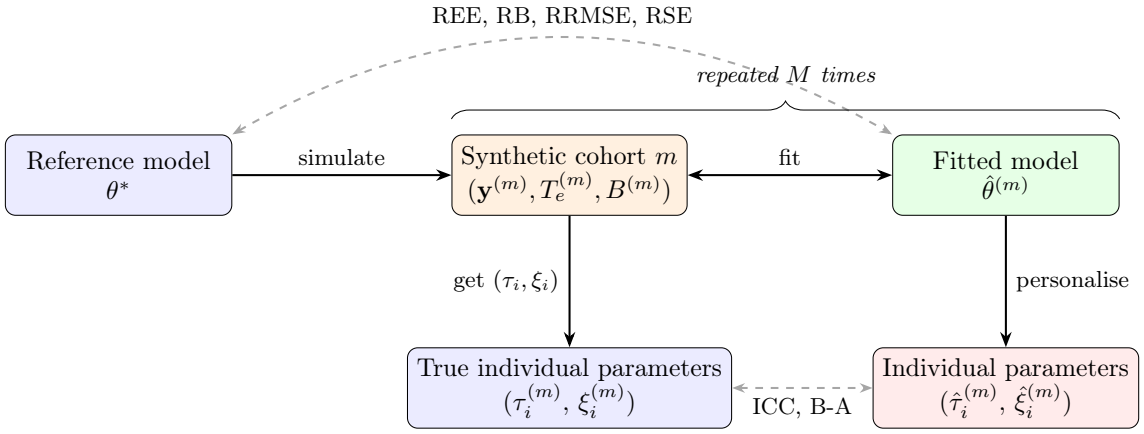

Figure 2: Schematic of the simulation-based evaluation pipeline. The reference model with fixed parameters θ^* generates M synthetic cohorts; for each cohort, a fresh model is fitted and then personalised.

Population parameter recovery. Let $\theta \in \mathbb{R}$ denote a scalar estimand, a single component of θ^*, and let $\hat{\theta}^{(m)}$ be its estimate from repetition m, for $m = 1, \ldots, M$. All relative quantities below are expressed as percentages when they derive from REE. The parameters evaluated with these metrics include the population distributions of the individual random effects (t_0, σ_τ, σ_ξ), the noise (σ), the average trajectory parameters (g, v_0), and the Weibull survival parameters (ν, ρ).

The Relative Estimation Error for repetition m is:

$$\mathrm{REE}(m) = \frac{\hat{\theta}^{(m)} - \theta}{|\theta|} \times 100, \tag{8}$$

and measures the signed percentage deviation of a single estimate from the true value. Its distribution across repetitions reflects the full sampling behaviour of the estimator at sample size N.

The Relative Bias (RB) is the mean REE over all repetitions:

$$\mathrm{RB}(\hat{\theta}) = \frac{1}{M} \sum_{m=1}^{M} \mathrm{REE}(m), \tag{9}$$

and quantifies the systematic tendency of the estimator to over- or under-estimate θ across repeated datasets of the same size.

The Relative Root Mean Squared Error (RRMSE) combines bias and variance into a single scalar:

$$\mathrm{RRMSE}(\hat{\theta}) = \sqrt{\frac{1}{M} \sum_{m=1}^{M} \mathrm{REE}(m)^2}. \tag{10}$$

It reduces to RB when the estimator is unbiased, and always satisfies RRMSE $\geq |\mathrm{RB}|$, so any excess of RRMSE over |RB| is attributable to Monte Carlo variability.

The Empirical Relative Standard Error (RSE) measures the variability of the estimator relative to its own mean, independently of the true value:

$$\mathrm{RSE}(\hat{\theta}) = \frac{\mathrm{SE}_{\mathrm{emp}}(\hat{\theta})}{|\bar{\hat{\theta}}|}, \qquad \mathrm{SE}_{\mathrm{emp}}(\hat{\theta}) = \sqrt{\frac{\sum_{m=1}^{M} \left(\hat{\theta}^{(m)} - \bar{\hat{\theta}}\right)^2}{M-1}}, \tag{11}$$

where $\bar{\hat{\theta}} = M^{-1} \sum_m \hat{\theta}^{(m)}$. The RSE is a dimensionless coefficient of variation for the estimator: values close to zero indicate a stable estimator whose output changes little from one synthetic cohort to the next, while large values indicate that the estimation is sensitive to the particular realization of the data.

Individual random effect recovery. Once the population parameters are estimated, individual random effects $(\hat{\tau}_i, \hat{\xi}_i)$ are recovered by personalisation. Since the population parameters β and $\boldsymbol{\zeta}$ are linear in the latent sources $\mathbf{s}_i$, they are not identifiable in the absolute sense: any rotation of the source space produces an equivalent model with identical likelihood. Therefore, the evaluation focuses on the products $\mathbf{w}_i = \mathbf{A}\mathbf{s}_i$ and $(\boldsymbol{\zeta}^\top \mathbf{s}_i)_l$, which represent the actual space and survival shifts that enter the model's generative mechanism and are considered as individual random effects. The true and estimated values of these quantities are compared across all N patients and M repetitions using the Intraclass Correlation Coefficient and the Bland-Altman analysis.

The Intraclass Correlation Coefficient ICC(3,1), defined under the two-way mixed-effects model with $k = 2$ raters and absolute agreement, is:

$$\mathrm{ICC}(3,1) = \frac{MS_B - MS_E}{MS_B + (k-1)\, MS_E}, \quad k = 2, \tag{12}$$

where MS_B is the mean square between subjects and MS_E is the mean square error, computed from the matrix whose two columns are the true and estimated values. The ICC(3,1) is the preferred agreement statistic here rather than the Pearson correlation, because Pearson's r is invariant to linear rescaling: an estimator that correctly ranks individuals but applies a systematic affine shift would obtain a high r while still being practically misleading. The ICC(3,1) penalises both lack of correlation and systematic offset, so that an ICC of 1 indicates exact pointwise agreement, an ICC near 0 indicates agreement no better than chance, and negative values reveal a systematic disagreement that is worse than ignoring the model entirely. Reporting ICC(3,1) alongside Pearson's r therefore makes it possible to distinguish a correctly ordered but shifted estimator from one that achieves genuine agreement.

The Bland-Altman analysis provides a graphical complement to the ICC. For each individual i and each repetition m, the difference $\hat{\theta}_i^{(m)} - \theta_i^{(m)}$ is plotted against the mean $(\hat{\theta}_i^{(m)} + \theta_i^{(m)})/2$, pooled across all M repetitions. Three summary quantities are reported: the bias (mean difference), which reveals any constant over- or under-estimation; the limits of agreement $\pm 1.96\,\mathrm{SD}$, which delimit the interval capturing approximately 95% of individual differences; and the dependence of the difference on the mean, which would indicate proportional bias. A narrow symmetric band centred on zero is the signature of an accurate and unbiased personalisation procedure.

6.2 Experimental Setup

présenter les features (dire à quels indices elles correspondent), présenter un peu le dataset

The reference model θ^* is a joint model fitted on the PRO-ACT dataset, with a single event ($L = 1$). The simulation study is carried out in the *random* visit mode with the following study parameters: first-visit offset $\bar{\delta}_f = -2.0$ years and $\sigma_{\delta_f} = 1.0$ year; follow-up duration $\overline{T_f} = 6.0$ years and $\sigma_{T_f} = 2.0$ years; inter-visit interval $\bar{\delta}_v = 0.083$ years and $\sigma_{\delta_v} = 0.042$ years; and a minimum spacing between visits of 0.05 years. These parameters are deliberately chosen to place each simulated cohort in favourable conditions: the short inter-visit interval and long follow-up produce a dense observation schedule, so that each patient contributes many data points. This abundance of information per individual reduces the number of MCMC-SAEM iterations required to reach convergence both at the fitting and at the personalisation stage.

Each synthetic cohort contains $N = 1000$ patients, and the experiment is repeated $M = 50$ times. Each fresh model is fitted using the MCMC-SAEM algorithm with $N_{\text{iter}} = 6000$ iterations, and the subsequent personalisation step runs for $N_{\text{perso}} = 2000$ iterations. Naturally, the computational cost of this procedure is substantial, as it requires fitting M separate joint models from scratch. Better results could be obtained by increasing N and M, but this would further increase the computational burden. The chosen values of N and M represent a compromise between statistical power and practical feasibility, while still providing a robust assessment of the model's self-consistency.

6.3 Results and Interpretation

6.3.1 Population parameter recovery

TODO + mettre plus de lignes dans les figures

Table 1 and Figure 3 summarise the results of the population parameter recovery.

Table 1: Population parameter recovery metrics.

Parameter	θ^* (reference)	$\bar{\hat{\theta}}$ (estimate)	RB (%)	RRMSE (%)	RSE
$g^{(0)}$	10.461	9.595	−8.27 [−9.41, −7.23]	11.41 [10.60, 12.28]	0.086 [0.076, 0.095]
$g^{(1)}$	2.326	2.122	−8.80 [−10.03, −7.52]	12.61 [11.55, 13.66]	0.099 [0.087, 0.111]
$g^{(2)}$	1.951	1.803	−7.60 [−8.71, −6.52]	11.16 [10.26, 12.06]	0.089 [0.079, 0.098]
$g^{(3)}$	3.415	3.213	−5.92 [−6.68, −5.17]	8.09 [7.46, 8.76]	0.059 [0.052, 0.066]
ρ	1.849	1.851	0.10 [−0.61, 0.81]	5.01 [4.50, 5.51]	0.050 [0.045, 0.055]
$v_0^{(0)}$	0.078	0.083	6.46 [5.32, 7.65]	10.53 [9.56, 11.53]	0.078 [0.070, 0.086]
$v_0^{(1)}$	0.257	0.263	2.23 [1.54, 2.93]	5.57 [5.05, 6.12]	0.050 [0.045, 0.054]
$v_0^{(2)}$	0.238	0.241	1.12 [0.52, 1.74]	4.52 [4.12, 4.92]	0.043 [0.039, 0.047]
$v_0^{(3)}$	0.133	0.135	2.05 [1.37, 2.73]	5.25 [4.75, 5.78]	0.048 [0.043, 0.052]
ν	4.047	4.142	2.33 [1.64, 3.00]	5.43 [4.97, 5.91]	0.048 [0.043, 0.053]
$\sigma^{(0)}$	0.069	0.065	−4.60 [−4.76, −4.45]	4.73 [4.58, 4.89]	0.012 [0.011, 0.013]
$\sigma^{(1)}$	0.079	0.075	−4.16 [−4.28, −4.04]	4.25 [4.14, 4.38]	0.009 [0.009, 0.010]
$\sigma^{(2)}$	0.074	0.072	−3.08 [−3.17, −2.97]	3.17 [3.07, 3.27]	0.008 [0.007, 0.009]
$\sigma^{(3)}$	0.045	0.044	−1.26 [−1.37, −1.15]	1.47 [1.37, 1.56]	0.008 [0.007, 0.009]
t_0	56.699	56.948	0.44 [0.35, 0.53]	0.83 [0.76, 0.90]	0.007 [0.006, 0.008]
σ_τ	11.630	11.639	0.08 [−0.24, 0.38]	2.32 [2.10, 2.54]	0.023 [0.021, 0.025]
σ_ξ	1.078	1.079	0.03 [−0.27, 0.37]	2.32 [2.11, 2.56]	0.023 [0.021, 0.026]

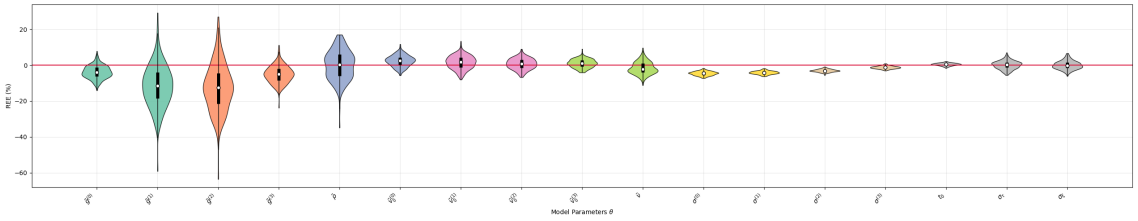

Figure 3: Relative error of estimation (REE) for all parameters.

6.3.2 Individual random effect recovery

Table 2: Individual random effect recovery metrics.

Parameter	ICC(3,1)	Pearson's r
ξ_i	0.988 ± 0.004	0.989 ± 0.004
τ_i	0.995 ± 0.002	0.995 ± 0.002
$w_{i,0}$	0.989 ± 0.003	0.990 ± 0.003
$w_{i,1}$	0.990 ± 0.003	0.991 ± 0.003
$w_{i,2}$	0.982 ± 0.011	0.991 ± 0.003
$\boldsymbol{\zeta}^\top \mathbf{s}_i$	0.965 ± 0.027	0.978 ± 0.013

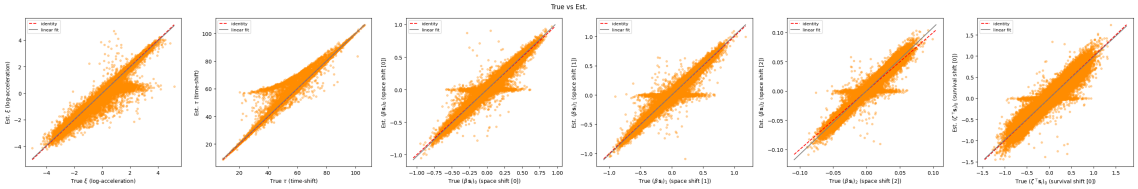

Figure 4: Scatter plots of estimated vs. true individual parameters across all repetitions. The dashed line indicates the identity line of perfect agreement.

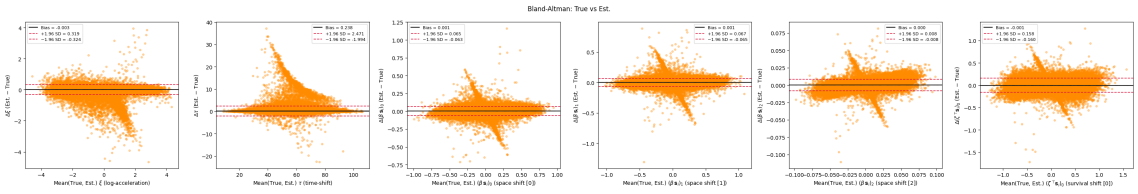

Figure 5: Bland-Altman plots for ξ (left) and τ (right). The solid line indicates the bias, and the dashed lines indicate the limits of agreement.

7 CS ONLY: Ethical and Sustainability Considerations

7.1 Simulation as an Ethical Response to Patient Data Privacy

A recurring concern throughout the internship was the handling of patient data from neurodegenerative disease cohorts. The datasets used to calibrate the reference models, such as PRO-ACT for amyotrophic lateral sclerosis, contain sensitive longitudinal health information collected from individuals suffering from severe and often fatal conditions. Access to these datasets is governed by strict ethical protocols, data use agreements, and institutional review board approvals, and rightly so: the patients who consented to share their clinical trajectories did so in the expectation that their data would be used responsibly and that their privacy would be protected.

This constraint has direct consequences for research practice. Sharing fitted model parameters or reproducing a published result often requires access to the original dataset, which may not be redistributable. Collaborative method development across institutions is slowed by the administrative and legal overhead of data transfer agreements. Reviewers of a submitted paper cannot, in general, verify the reported results on the same data. These are not merely logistical inconveniences: they represent a structural tension between the scientific norm of reproducibility and the ethical imperative of data protection.

The simulation tools developed during this internship offer a partial but concrete resolution to this tension. A synthetic cohort generated under a calibrated model preserves the statistical properties of the original population, including trajectory shapes, individual variability, censoring patterns, and event rates, while containing no identifiable patient information whatsoever. No simulated individual corresponds to any real patient; the entire dataset is a probabilistic artefact of the fitted model. Such synthetic datasets can therefore be freely distributed alongside published code, included in software documentation, used in teaching workshops, and shared across institutions without any risk of re-identification or breach of consent.

During the internship, I came to appreciate that this property is not a secondary convenience but also an ethical contribution. The workshop organised by the laboratory on longitudinal data analysis with Leaspy relied entirely on simulated datasets, allowing participants to engage with realistic disease progression scenarios without any exposure to real patient data. More broadly, the availability of high-quality synthetic data lowers the barrier to entry for researchers who wish to develop or benchmark new methods but do not have direct access to clinical cohorts, thereby contributing to a more equitable and open research ecosystem.

This perspective also made me more attentive to the responsibilities that accompany the generation of synthetic data. A simulated dataset that faithfully reproduces the statistical structure of a real cohort could, in principle, be mistaken for real data or used to make clinical claims without the safeguards that govern actual patient studies. Ensuring that synthetic datasets are clearly labelled, that their provenance is documented, and that their limitations are explicitly stated is therefore an ethical obligation that accompanies the technical capability of simulation. In the Leaspy codebase, this concern was addressed through documentation and metadata conventions that distinguish simulated outputs from real observations.

7.2 Computational Cost and Environmental Responsibility

The validation study reported in this work required fitting several separate joint models from scratch, each involving a lot of MCMC-SAEM iterations on a cohort of multiple synthetic patients, followed by a personalisation step which was also iterative per patient. The total computational budget amounted to several hours of continuous execution on institutional computing infrastructure. During the development phase, numerous additional runs were performed for debugging, hyperparameter exploration, and convergence diagnostics, multiplying the effective resource consumption well beyond the final reported experiment.

This experience confronted me with the environmental cost of simulation-intensive research. Each model fit consumes electrical energy, generates heat, and contributes to the carbon footprint of the research activity. While the individual energy consumption of a single MCMC-SAEM run is modest compared to, for instance, large-scale deep learning training, the cumulative effect of hundreds of exploratory and validation runs over a six-month internship is not negligible.

Several practical measures were adopted to mitigate this cost. First, the dense observation schedule used in the simulation study was deliberately chosen not only for statistical reasons but also because it reduces the number of SAEM iterations required for convergence, thereby shortening each run. Second, preliminary experiments on smaller cohorts were systematically conducted before committing to the full-scale study, in order to detect implementation errors or convergence failures early and avoid wasting computational resources on flawed configurations. Third, intermediate results and fitted model objects were systematically saved to disk, so that a failed or interrupted run could be resumed or analysed without repeating the entire computation. These practices, while individually modest, collectively reduced the total number of unnecessary computations by a meaningful factor.

7.3 Open-Source Software for Clinical Research: Reliability and Responsibility

Leaspy is an open-source library intended to be used not only by its developers but also by applied researchers, clinicians, and students who may not have the statistical or computational expertise to audit the implementation in detail. During the internship, I contributed to several aspects of the software beyond the core simulation pipeline: reviewing pull requests, writing and improving unit and functional tests, and strengthening the documentation. These activities, which occupied a substantial fraction of the internship time,

brought me into direct contact with the question of software responsibility in a clinical research context.

The stakes are significant. A bug in the simulation algorithm could produce synthetic datasets whose statistical properties do not match the intended model, leading to incorrect conclusions in a validation study. An error in the `estimate` method could bias the survival predictions used for patient stratification. A poorly documented function could be misused by a researcher unfamiliar with the model assumptions, producing results that are technically valid but scientifically misleading. In all these cases, the consequences extend beyond the software itself: they affect the scientific conclusions drawn from the software and, ultimately, the clinical decisions informed by those conclusions.

This awareness shaped my approach to software development throughout the internship. Every new feature was accompanied by unit tests designed to catch regressions, and functional tests that verified end-to-end consistency against known reference values. Code reviews were conducted not only for correctness but also for clarity, because a piece of code that is correct but opaque is a latent source of future errors when modified by another contributor.

This dimension of the internship was formative in a way that I had not anticipated. My academic training in statistics and machine learning had emphasized the importance of methodological rigour, but had given comparatively little attention to the software practices that translate methodological ideas into reliable, usable tools. Working on Leaspy taught me that the gap between a correct mathematical formulation and a correct software implementation is wide, and that bridging it requires a distinct set of skills: systematic testing, defensive programming, clear documentation, and a culture of peer review. These are not peripheral concerns but core competencies for any researcher whose work involves computational methods, and I expect them to remain central to my professional practice in the years ahead.

8 TBD: Preliminary Work on the Phd Subject

TODO

9 Conclusion

This internship addressed a central methodological and software challenge for the Leaspy library: providing simulation and evaluation tools for complex non-linear mixed-effects models, and more specifically for the joint model linking longitudinal and survival data. In a context where real clinical datasets never provide access to the true latent disease stage or to the true individual random effects, simulation constitutes an indispensable instrument both for validating statistical procedures and for testing software implementations under controlled conditions.

The main contribution of this work was the development and integration of a simulation pipeline for the joint model within the Leaspy codebase. This required extending the existing logic of the library to support the generation of event times alongside longitudinal trajectories, handling censoring constraints, adapting the `estimate` method to return survival-related quantities, and generalizing the framework to competing events. These developments made it possible to simulate synthetic cohorts in a way that is coherent with the mathematical structure of the joint model and with the design principles of the Leaspy API.

A second important contribution was the implementation of a simulation-based validation strategy aimed at evaluating the self-consistency of the full pipeline. By repeatedly simulating datasets from a reference fitted model and re-estimating the parameters, the study assessed the recovery of both population parameters and personalized individual random effects. The results show that the main longitudinal parameters and the subject-level time-shift and acceleration effects are recovered with good accuracy in the considered setting. At the same time, some parameters related to survival and latent-source couplings exhibit substantially larger estimation errors, highlighting the practical difficulties of identifying second-order effects in moderately sized cohorts. These findings are informative not only for the specific implementation developed during the internship, but also more generally for the statistical use of the joint model.

Beyond the work described in detail in this report, the internship also involved a broader contribution to the life of the project: reviewing pull requests, strengthening the test suite with unit and functional tests, improving parts of the documentation, and preparing and leading a workshop on longitudinal data, with a particular focus on simulation. These activities were essential to ensure that the developed features are not only methodologically relevant, but also maintainable, documented and usable by other contributors and end users.

Overall, this internship provided an opportunity to work at the interface between statistics, machine learning and scientific software engineering, in a highly interdisciplinary research environment. It also illustrated a key aspect of modern methodological research: proposing a model is not sufficient; one must also provide the computational tools, validation procedures and software standards that make its use reliable and reproducible. In that respect, the work carried out during these six months contributes to making Leaspy a more robust and more complete platform for longitudinal disease progression modeling.

Several perspectives naturally follow from this work. On the methodological side, future simulation studies could investigate more systematically the effect of sample size, censoring intensity, visit sparsity, or model misspecification on parameter recovery. On

the software side, the simulation framework could be extended further, for instance by allowing more flexible visit-generation schemes, richer noise models, or closer alignment between continuous-time event generation and the current grid-based implementation. More broadly, the tools developed here may support future benchmarking studies, educational uses, and the generation of privacy-preserving synthetic datasets for open research. These perspectives make simulation not only a technical feature of Leaspy, but also a strategic component of its future development.

References

[1] Irene Y Chen, Rahul G Krishnan, and David Sontag. Clustering interval-censored time-series for disease phenotyping. In *Proceedings of the AAAI conference on artificial intelligence*, volume 36, pages 6211–6221, 2022.

[2] Arman Eshaghi, Alexandra L Young, Peter A Wijeratne, Ferran Prados, Douglas L Arnold, Sridar Narayanan, Charles RG Guttmann, Frederik Barkhof, Daniel C Alexander, Alan J Thompson, et al. Identifying multiple sclerosis subtypes using unsupervised machine learning and mri data. *Nature communications*, 12(1):2078, 2021.

[3] Hubert M Fonteijn, Marc Modat, Matthew J Clarkson, Josephine Barnes, Manja Lehmann, Nicola Z Hobbs, Rachael I Scahill, Sarah J Tabrizi, Sebastien Ourselin, Nick C Fox, et al. An event-based model for disease progression and its application in familial alzheimer's disease and huntington's disease. *NeuroImage*, 60(3):1880–1889, 2012.

[4] Bruno M Jedynak, Andrew Lang, Bo Liu, Elyse Katz, Yanwei Zhang, Bradley T Wyman, David Raunig, C Pierre Jedynak, Brian Caffo, Jerry L Prince, et al. A computational neurodegenerative disease progression score: method and results with the alzheimer's disease neuroimaging initiative cohort. *Neuroimage*, 63(3):1478–1486, 2012.

[5] Sofia Kaisaridi, Juliette Ortholand, Caglayan Tuna, Nicolas Gensollen, and Sophie Tezenas Du Montcel. A mixture model for subtype identification: Application to cadasil. In *ISCB 46-46th Annual Conference of the International Society for Clinical Biostatistics*, 2025.

[6] Igor Koval, Alexandre Bône, Maxime Louis, Thomas Lartigue, Simona Bottani, Arnaud Marcoux, Jorge Samper-Gonzalez, Ninon Burgos, Benjamin Charlier, Anne Bertrand, et al. Ad course map charts alzheimer's disease progression. *Scientific Reports*, 11(1):8020, 2021.

[7] Igor Koval, Thomas Dighiero-Brecht, Allan J Tobin, Sarah J Tabrizi, Rachael I Scahill, Sophie Tezenas du Montcel, Stanley Durrleman, and Alexandra Durr. Forecasting individual progression trajectories in huntington disease enables more powered clinical trials. *Scientific Reports*, 12(1):18928, 2022.

[8] Estelle Kuhn and Marc Lavielle. Maximum likelihood estimation in nonlinear mixed effects models. *Computational statistics & data analysis*, 49(4):1020–1038, 2005.

[9] Dan Li, Samuel Iddi, Wesley K Thompson, Michael C Donohue, and Alzheimer's Disease Neuroimaging Initiative. Bayesian latent time joint mixed effect models for multicohort longitudinal data. *Statistical methods in medical research*, 28(3):835–845, 2019.

[10] Marco Lorenzi, Maurizio Filippone, Giovanni B Frisoni, Daniel C Alexander, Sébastien Ourselin, Alzheimer's Disease Neuroimaging Initiative, et al. Probabilistic disease

progression modeling to characterize diagnostic uncertainty: application to staging and prediction in alzheimer's disease. *NeuroImage*, 190:56–68, 2019.

[11] Etienne Maheux, Igor Koval, Juliette Ortholand, Colin Birkenbihl, Damiano Archetti, Vincent Bouteloup, Stéphane Epelbaum, Carole Dufouil, Martin Hofmann-Apitius, and Stanley Durrleman. Forecasting individual progression trajectories in alzheimer's disease. *Nature Communications*, 14(1):761, 2023.

[12] Răzvan V Marinescu, Neil P Oxtoby, Alexandra L Young, Esther E Bron, Arthur W Toga, Michael W Weiner, Frederik Barkhof, Nick C Fox, Polina Golland, Stefan Klein, et al. Tadpole challenge: Accurate alzheimer's disease prediction through crowdsourced forecasting of future data. In *International workshop on predictive intelligence in medicine*, pages 1–10. Springer, 2019.

[13] Tim P Morris, Ian R White, and Michael J Crowther. Using simulation studies to evaluate statistical methods. *Statistics in medicine*, 38(11):2074–2102, 2019.

[14] Juliette Ortholand, Stanley Durrleman, and Sophie Tezenas du Montcel. A joint spatiotemporal model for multiple longitudinal markers and competing events. *arXiv preprint arXiv:2501.08960*, 2025.

[15] Juliette Ortholand, Nicolas Gensollen, Stanley Durrleman, and Sophie Tezenas Du Montcel. Joint model with latent disease age: Overcoming the need for reference time. *Statistical Methods in Medical Research*, 35(2):225–239, 2026.

[16] Neil P Oxtoby. Data-driven disease progression modeling. *Machine learning for brain disorders*, pages 511–532, 2023.

[17] Jean-Baptiste Schiratti, Stéphanie Allassonnière, Olivier Colliot, and Stanley Durrleman. A bayesian mixed-effects model to learn trajectories of changes from repeated manifold-valued observations. *Journal of Machine Learning Research*, 18(133):1–33, 2017.

[18] Vikram Venkatraghavan, Esther E Bron, Wiro J Niessen, Stefan Klein, Alzheimer's Disease Neuroimaging Initiative, et al. Disease progression timeline estimation for alzheimer's disease using discriminative event based modeling. *NeuroImage*, 186:518–532, 2019.

[19] Jacob W Vogel, Alexandra L Young, Neil P Oxtoby, Ruben Smith, Rik Ossenkoppele, Olof T Strandberg, Renaud La Joie, Leon M Aksman, Michel J Grothe, Yasser Iturria-Medina, et al. Four distinct trajectories of tau deposition identified in alzheimer's disease. *Nature medicine*, 27(5):871–881, 2021.

[20] Alexandra L Young, Razvan V Marinescu, Neil P Oxtoby, Martina Bocchetta, Keir Yong, Nicholas C Firth, David M Cash, David L Thomas, Katrina M Dick, Jorge Cardoso, et al. Uncovering the heterogeneity and temporal complexity of neurodegenerative diseases with subtype and stage inference. *Nature communications*, 9(1):4273, 2018.

[21] Alexandra L Young, Neil P Oxtoby, Sara Garbarino, Nick C Fox, Frederik Barkhof, Jonathan M Schott, and Daniel C Alexander. Data-driven modelling of neurodegenerative disease progression: thinking outside the black box. *Nature Reviews Neuroscience*, 25(2):111–130, 2024.